

HDOC Day-33

Aim

To write a menu-driven C program using separate functions to verify the associative properties of matrix addition and matrix multiplication using variable-length 2D integer arrays.

Problem Statement

1. Read three matrices A, B and C of dimensions (m x n) and verify:

$$(A + B) + C = A + (B + C)$$

2. Read matrices A (m x n), B (n x p) and C (p x q) and verify:

$$(A \times B) \times C = A \times (B \times C)$$

3. Exit.

Algorithm

1. Display the menu and read the user's choice.
2. For addition, read m and n, then read A, B and C. Compute both sides using an addition function and compare them.
3. For multiplication, read m, n, p and q, then read A, B and C. Compute A x B and B x C, then compute both sides and compare them.
4. Display whether the associative property is verified.
5. Repeat until Exit is selected.

C Program

```
#include <stdio.h>

void readMatrix(int r, int c, int a[r][c]) {
    int i, j;
    for (i = 0; i < r; i++)
        for (j = 0; j < c; j++)
            scanf("%d", &a[i][j]);
}

void addMatrix(int r, int c, int a[r][c], int b[r][c],
               int sum[r][c]) {
    int i, j;
    for (i = 0; i < r; i++)
        for (j = 0; j < c; j++)
            sum[i][j] = a[i][j] + b[i][j];
}

void multiplyMatrix(int r1, int c1, int c2,
                   int a[r1][c1], int b[c1][c2],
                   int result[r1][c2]) {
    int i, j, k;
    for (i = 0; i < r1; i++)
        for (j = 0; j < c2; j++) {
            result[i][j] = 0;
            for (k = 0; k < c1; k++)
                result[i][j] += a[i][k] * b[k][j];
        }
}

int equalMatrix(int r, int c, int a[r][c], int b[r][c]) {
    int i, j;
    for (i = 0; i < r; i++)
        for (j = 0; j < c; j++)
            if (a[i][j] != b[i][j])
                return 0;
    return 1;
}

int main() {
```

```

int choice;

do {
    printf("\n1. Associative property of addition");
    printf("\n2. Associative property of multiplication");
    printf("\n3. Exit");
    printf("\nEnter choice: ");
    scanf("%d", &choice);

    if (choice == 1) {
        int m, n;
        printf("Enter rows and columns: ");
        scanf("%d%d", &m, &n);

        int A[m][n], B[m][n], C[m][n];
        int AB[m][n], BC[m][n];
        int left[m][n], right[m][n];

        printf("Enter A:\n"); readMatrix(m, n, A);
        printf("Enter B:\n"); readMatrix(m, n, B);
        printf("Enter C:\n"); readMatrix(m, n, C);

        addMatrix(m, n, A, B, AB);
        addMatrix(m, n, AB, C, left);
        addMatrix(m, n, B, C, BC);
        addMatrix(m, n, A, BC, right);

        if (equalMatrix(m, n, left, right))
            printf("Verified: (A+B)+C = A+(B+C)\n");
        else
            printf("Not verified.\n");
    }

    else if (choice == 2) {
        int m, n, p, q;
        printf("Enter m n p q: ");
        scanf("%d%d%d%d", &m, &n, &p, &q);

        int A[m][n], B[n][p], C[p][q];
        int AB[m][p], BC[n][q];
        int left[m][q], right[m][q];

        printf("Enter A:\n"); readMatrix(m, n, A);
        printf("Enter B:\n"); readMatrix(n, p, B);
        printf("Enter C:\n"); readMatrix(p, q, C);

        multiplyMatrix(m, n, p, A, B, AB);
        multiplyMatrix(m, p, q, AB, C, left);
        multiplyMatrix(n, p, q, B, C, BC);
        multiplyMatrix(m, n, q, A, BC, right);

        if (equalMatrix(m, q, left, right))
            printf("Verified: (A*B)*C = A*(B*C)\n");
        else
            printf("Not verified.\n");
    }

    else if (choice != 3)
        printf("Invalid choice.\n");

} while (choice != 3);

return 0;
}

```

Sample Result

For valid matrices, the program displays **Verified: (A+B)+C = A+(B+C)** for addition and **Verified: (A*B)*C = A*(B*C)** for multiplication.

Conclusion

Thus, the C program verifies the associative properties of matrix addition and matrix multiplication using separate functions and variable-length 2D arrays.